



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

Comparativa implementación Asistente de Voz: Local VS Cloud

TRABAJO HERRAMIENTAS Y APLICACIONES DE LA INTELIGENCIA ARTIFICIAL

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Andrés Pacheco,
Miquel Gómez

CAPÍTULO 1

Introducción

En el contexto de la vida cotidiana, los asistentes de voz se han convertido en herramientas muy usadas por su gran versatilidad. Desde los primeros *Hey Siri*, a los *Ok Google* o *Alexa*, los asistentes virtuales han llegado a multitud de hogares y ayudan en el día a día de diversas tareas.

Al principio, estos asistentes eran muy limitados y no tenían apenas capacidad de respuesta. Eran todo instrucciones predefinidas que seguían casi como `if else`. Las voces y las frases que estas podían decir estaban pre-grabadas, por lo que no se podían salir de ese pequeño conjunto de respuestas.

Hoy en día, esto ha cambiado bastante. De primeras, la llegada de los modelos de lenguaje, capaces de generalizar las respuestas que puede dar un sistema, soluciona las limitaciones de *'a qué puede contestar el sistema'*, ahora es prácticamente a todo. Luego, si se le suma que los modelos de voz modernos son capaces de pronunciar cualquier frase, palabra o fonema en casi todos los idiomas, se tiene la receta perfecta para crear un sistema que responda a cualquier petición en segundos.

En este trabajo, se pretende abordar la creación de un sistema de preguntas y respuestas mediante un asistente de voz virtual. En su conjunto, el proyecto consiste en una investigación para analizar la viabilidad de desarrollar un asistente de voz propio, evaluando y confrontando el rendimiento, la facilidad de desarrollo y la complejidad de utilizar alternativas totalmente locales frente a soluciones basadas en modelos de diferentes proveedores *cloud* (en la nube) para cada una de las fases (transcripción, RAG con información local y web, y síntesis de voz).

A lo largo de las secciones se habla de la estructura que se ha seguido, con qué intenciones, con qué tecnologías se ha implementado y cómo se ha ajustado.

Uso de herramientas LLMs

Para el desarrollo de este trabajo, se ha hecho uso de herramientas LLM en diferentes puntos. Primero, para la interfaz gráfica, se ha partido de una plantilla y se ha usado Copilot para que realice la integración con el resto de la app (archivo `app/app.py`). Luego, también se ha usado Claude Code, muy útil a la hora de resolver conflictos entre versiones. Por último, para revisar la redacción de esta memoria, se ha hecho uso de Gemini.

Todo se ha hecho con la supervisión de los autores y con control total sobre lo que hacían estas herramientas.

Hardware utilizado

Los experimentos y la demo se han lanzado en una máquina personal: GPU NVIDIA RTX 5070 (12 GB VRAM), procesador AMD Ryzen 9 9000X, 64 GB de RAM y Ubuntu 24.04 LTS.

Además, todas las pruebas cloud se realizaron en planes gratuitos sin alcanzar el límite de uso.

CAPÍTULO 2

Estado del Arte

Actualmente, el panorama de los asistentes de voz se basa mucho en modelos de lenguaje de gran tamaño (LLMs) y sistemas de síntesis de voz (TTS) que han evolucionado enormemente en los últimos años.

Por un lado, en el entorno *cloud*, el estado del arte (SOTA) lo lideran los modelos multimodales nativos, como GPT-4o de OpenAI [1] o Gemini Live [2]. La revolución de estos sistemas es que prescindan de la clásica cascada (pasar de voz a texto, procesar y generar voz); en su lugar, procesan y generan el audio directamente. Esto los hace rapidísimos y muy naturales. Sin embargo, tienen pegas importantes: son "cajas negras", obligan a enviar datos privados a servidores de terceros, tienen costes por uso, y montarles un RAG propio profundo y flexible no es tan sencillo.

Por otro lado, en el entorno *local* y *open-source*, el estado del arte sigue apostando por la arquitectura modular (STT → LLM → TTS), que es exactamente la que se emplea en este proyecto. Herramientas como Whisper [3] dominan la transcripción; modelos como Llama 3.1 [4] o Mistral-Nemo [5] permiten un razonamiento excelente en hardware doméstico; y sintetizadores ligeros como Kokoro [6] o Qwen3-TTS [7] logran voces fluidas en tiempo real consumiendo mínimos recursos.

Esto confirma que, aunque las grandes tecnológicas tiren por sistemas monolíticos en la nube, la arquitectura segmentada que usamos en este trabajo sigue siendo el estándar actual y necesario para crear asistentes privados, modulares y con RAG a medida [8]. Por ello, comparar el rendimiento de esta estructura en local frente a sus equivalentes en la nube es un ejercicio de total relevancia a día de hoy.

CAPÍTULO 3

Planteamiento del proyecto

Para poder implementar un asistente capaz de responder preguntas de forma confiable como el que se plantea, hay que tener en cuenta diversos puntos clave.

Primero que todo, el sistema debe ser cómodo de usar e interactuar con él. Para esto, la opción más intuitiva es que funcione mediante voz.

Si se piensa en qué situaciones se suelen usar estos asistentes, lo primero que viene a la mente son situaciones en las que no es posible hacer uso de tecnología, como al conducir, cocinar o incluso en la ducha. Son situaciones en las que se tienen las manos ocupadas y por lo tanto, no se puede hacer uso ni de dispositivos móviles, ni mucho menos de teclado y ratón. Por lo que hacer las preguntas mediante voz, es la solución perfecta.

Luego, este sistema tiene que poder interpretar la instrucción, lo que requiere de un sistema capaz de entender preguntas genéricas de cualquier ámbito. La tecnología que mejor responde a esta necesidad son los LLMs, no requieren de mayor presentación. Ahora, estos traen consigo algunos problemas por su misma naturaleza, haciendo que su uso no sea simple: estos modelos no tienen información de un contexto concreto, no tienen información actualizada del mundo en tiempo real, y pueden llegar a alucinar datos solo por querer darnos una respuesta.

Es así que la mejor forma de introducir un LLM en un sistema de este estilo, es mediante un RAG. Un RAG permite dotar al LLM de información específica de dicho contexto, y se puede generalizar su uso con cualquier tipo de documentos. Con esto, se soluciona el primer problema, pero aún quedaría el segundo. Para permitir que el sistema pueda tener datos actualizados, hay que darle otra herramienta: el acceso a internet, que será la segunda fuente de información fiable que tendrá el sistema. Con esto, estará provisto de todo lo necesario para responder a las preguntas.

Ahora bien, que pueda responder a las preguntas, no significa que las pueda hacer llegar al usuario. En este punto falta el último paso, que el asistente de verdad se comunique de vuelta. De nuevo, la forma de hacerlo es mediante voz, por lo que el sistema deberá implementar un sistema de voz o Text to Speech.

Con esto, el asistente estará dotado de la capacidad de responder a todo en tiempo real, con información fiable y actualizada, y todo ello mediante voz.

CAPÍTULO 4

Desarrollo

El sistema se divide en cuatro bloques (STT, RAG, búsqueda web y TTS). Se comentan en detalle únicamente esas secciones; se deja de lado la explicación de la búsqueda web, que es idéntica en ambas modalidades, basada en Tavily [9] y es cuestión una llamada MPC; y la interfaz gráfica, que usa Streamlit [10] y gestiona la interacción de forma abstracta: graba el audio del usuario, lo pasa al pipeline y reproduce la respuesta final en voz.

Los modelos se eligieron lo más equivalentes posible entre ambos entornos, exprimiendo al máximo las capacidades locales para evitar sesgos. Toda la comparativa parte además de la premisa de coste económico directo nulo.

4.1 Implementación Local

En la figura vemos la estructura del asistente de voz con modelos locales. El proceso es el siguiente: el usuario hace una pregunta por voz, esta se transcribe a texto con Whisper, se recupera el contexto relevante mediante RAG con Ollama, se evalúa la relevancia del contexto y se genera la respuesta, que finalmente se sintetiza a voz con Kokoro para comunicársela al usuario.

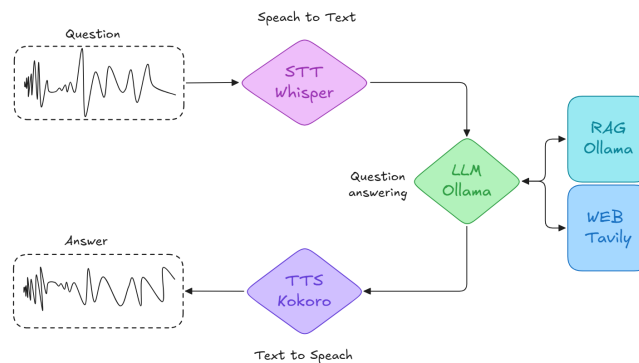


Figura 4.1: Estructura del asistente de voz con modelos *locales*.

4.1.1. Speech to Text (Whisper)

Se integró Whisper [3] en su variante large-v3, cargado directamente con la librería faster-whisper para aprovechar la cuantización INT8 y reducir la ocupación de VRAM. La llamada al modelo recibe el archivo de audio grabado desde la interfaz y devuelve la transcripción en texto plano, que se pasa directamente al módulo RAG.

El modelo funciona muy bien como se ve en los resultados más abajo. La integración fue sencilla y eficiente. Solo comentar que durante las pruebas se detectó una limitación práctica: el modelo

falla con siglas o nombres propios muy específicos del dominio ('IARFID' → 'MirFit'; 'PyCatan' → 'PaiKatan'). Estos errores se propagan en cadena, pues si la transcripción es incorrecta el RAG raramente localizará el documento correcto y la respuesta final queda comprometida.

4.1.2. RAG (Ollama)

Todo el componente RAG se orquestó con **LangChain** [8], aprovechando Ollama [11] como backend de inferencia local. Su uso se basa en introducir indexar, dar la transcripción, recuperar y generar la respuesta.

Indexación y persistencia

Los PDFs (nuestra información / contexto local) se cargan con PyPDFLoader y se trocean con RecursiveCharacterTextSplitter (chunk=1000 chars, overlap=200). Cada chunk se vectoriza con el modelo de embeddings **mistral-nemo** [5] servido por Ollama y se persiste en **ChromaDB**.

Esta parte es de las más importantes, ya que el rendimiento del RAG depende en gran medida de la calidad de la indexación. También por obvias razones de eficiencia, si la base de datos ya existe para evitar re-indexaciones innecesarias.

Evaluación de relevancia

Antes de que el LLM genere la respuesta final del usuario, se recuperan los $k = 10$ chunks más similares a la consulta y se le pasa ese contexto a **llama3.1** [4] con un prompt evaluador para que solo responda 'SIs' o 'NOs':

```
prompt_evaluador = ChatPromptTemplate.from_messages([
    ("system", "Eres un evaluador de relevancia. Responde estrictamente con una palabra: 'SI  
↪ ' o 'NO'."),
    ("human", f"Pregunta: {pregunta} \n\n Contexto recuperado: {documentos} \n\n ¿Contiene  
↪ este contexto la informacion necesaria para responder?")
])
```

Esto es únicamente para evaluar si el contexto local es suficiente o no para generar la respuesta. En caso de que el contexto local sea insuficiente o irrelevante, el sistema activa un mecanismo de *fallback* en que utiliza la herramienta **TavilySearchResults** [9] para obtener la información de la web. Para este caso, se ha establecido a 5 el número máximo de resultados que se devolverán por búsqueda y se ha indicado que se realice una búsqueda avanzada.

Generación de respuesta

Teniendo del contexto adecuado, en ambos casos, el prompt de la respuesta indica explícitamente al modelo a responder en texto plano sin Markdown, formato imprescindible para que la síntesis de voz no verbalice asteriscos ni guiones:

```
system_prompt = (
    "Eres un asistente de investigacion. Responde de forma natural, fluida y concisa, "  

    "no des detalles innecesarios y ve directamente al grano. "  

    f"La informacion proviene de {fuente_info}. "  

    "IMPORTANTE: No uses negritas, ni asteriscos, ni listas, ni ningun formato markdown. "  

    "Escribe todo en un parrafo o parrafos de texto plano, como si fuera una carta o un  

    ↪ mensaje de voz."  

    "\n\nContexto: {context}"
)
```

4.1.3. Text to Speech (Kokoro)

Una vez el cerebro del sistema ha generado la respuesta de texto, el último paso es comunicar el feedback al usuario mediante voz. Para ello, se integró.

Para la versión local se ha usado Kokoro [6]. Recibe el texto plano devuelto por el RAG (que debería estar limpio, sin formato), selecciona la voz que queramos y genera el audio directamente en memoria como array numpy, que la interfaz reproduce sin necesidad de escribir archivos intermedios en disco. El proceso completo corre en CPU/GPU de forma transparente según la disponibilidad del sistema.

4.2 Implementación Cloud

La implementación de local a cloud fue bastante directa. Se hizo uso de la misma lógica ya implementada en cada componente. Además, gracias a **LangChain**, al abstraer los proveedores bajo la misma interfaz, basta con cambiar la instancia del modelo para que el RAG completo funcionara, reutilizando los mismos prompts, la misma lógica de *fallback* y la misma estructura de orquestación.

La intención inicial era centralizar todos los servicios en **AWS**, pero problemas de validación de cuenta y suscripciones bloqueadas obligaron a diversificar en un collage de proveedores:

- **GROQ** [12]: Nos provee de una versión de Whisper igualita la usada en el entorno local. Además de poder acceder a algunos LLM como **llama-3.1-8b-instant** para hacer lo mismo que en el entorno local. Las llamadas se realizan mediante el wrapper ChatGroq de LangChain, lo que hizo el cambio completamente transparente a nivel de código.
- **Hugging Face** [13]: los embeddings para indexar todos los documentos se generan con `sentence-transformers` vía la clase `HuggingFaceEmbeddings`, sustituyendo directamente a los embeddings de Ollama en el paso de indexación y recuperación.
- **AWS Polly** [14]: se llama al servicio mediante boto3, para que devuelva un audio mp3 con el texto proporcionado. Todo super parecido a la integración de Kokoro.

Esta fragmentación introduce una complejidad operativa notable: cada proveedor gestiona su propia autenticación (API keys, credenciales AWS), cuotas de uso y condiciones de servicio, elevando el esfuerzo de despliegue respecto al entorno local. Toda la experimentación se realizó dentro de los planes gratuitos de cada proveedor, sin llegar al límite de uso en ningún caso. Ahora, esto es a fecha de Mayo del 2026, no se sabe en un futuro si alguno de los servicios podría cambiar sus condiciones o limitar el acceso, lo que es un riesgo inherente a depender de terceros.

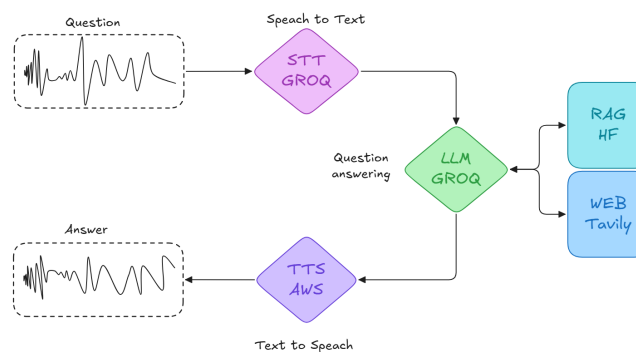


Figura 4.2: Estructura del asistente de voz con proveedores *cloud*.

CAPÍTULO 5

Resultados y Análisis

Una vez implementado todo el sistema en ambos entornos, se diseñó un experimento con 30 audios pregrabados, obteniendo 60 ejecuciones (30 por modalidad) para ver su rendimiento.

5.1 Metodología de Evaluación

- **Tiempos:** $T_{total} = T_{STT} + T_{RAG} + T_{TTS}$ (la búsqueda web se excluye por ser igual en ambas).
- **WER:** *Word Error Rate* sobre las transcripciones, normalizadas con `werpy.normalize` [15] (minúsculas, sin puntuación, espacios homogeneizados).
- **RAG Matching Source:** binario (0/1); el sistema usó la fuente correcta (local si el dato estaba en documentos, web si no).
- **RAG Matching Response:** binario (0/1); la respuesta final fue correcta independientemente de la fuente.
- **TTS Score (0–100):** valoración manual de fluidez, pausas lógicas y naturalidad.

5.2 Resultados

Tabla 5.1: Métricas obtenidas en la evaluación de los 30 audios.

Métrica	Local	Cloud
Transcribe Time (s)	3.95	0.41
RAG Time (s)	6.04	31.98
TTS Time (s)	1.03	0.86
Total Time (s)	11.02	33.25
Transcription WER (%)	5.22 %	6.32 %
RAG Matching Source (Raw)	24 / 30	27 / 30
RAG Matching Response (Raw)	19 / 30	25 / 30
RAG Matching Source (%)	80.00 %	90.00 %
RAG Matching Response (%)	63.33 %	83.33 %
TTS Score (0–100)	50	90

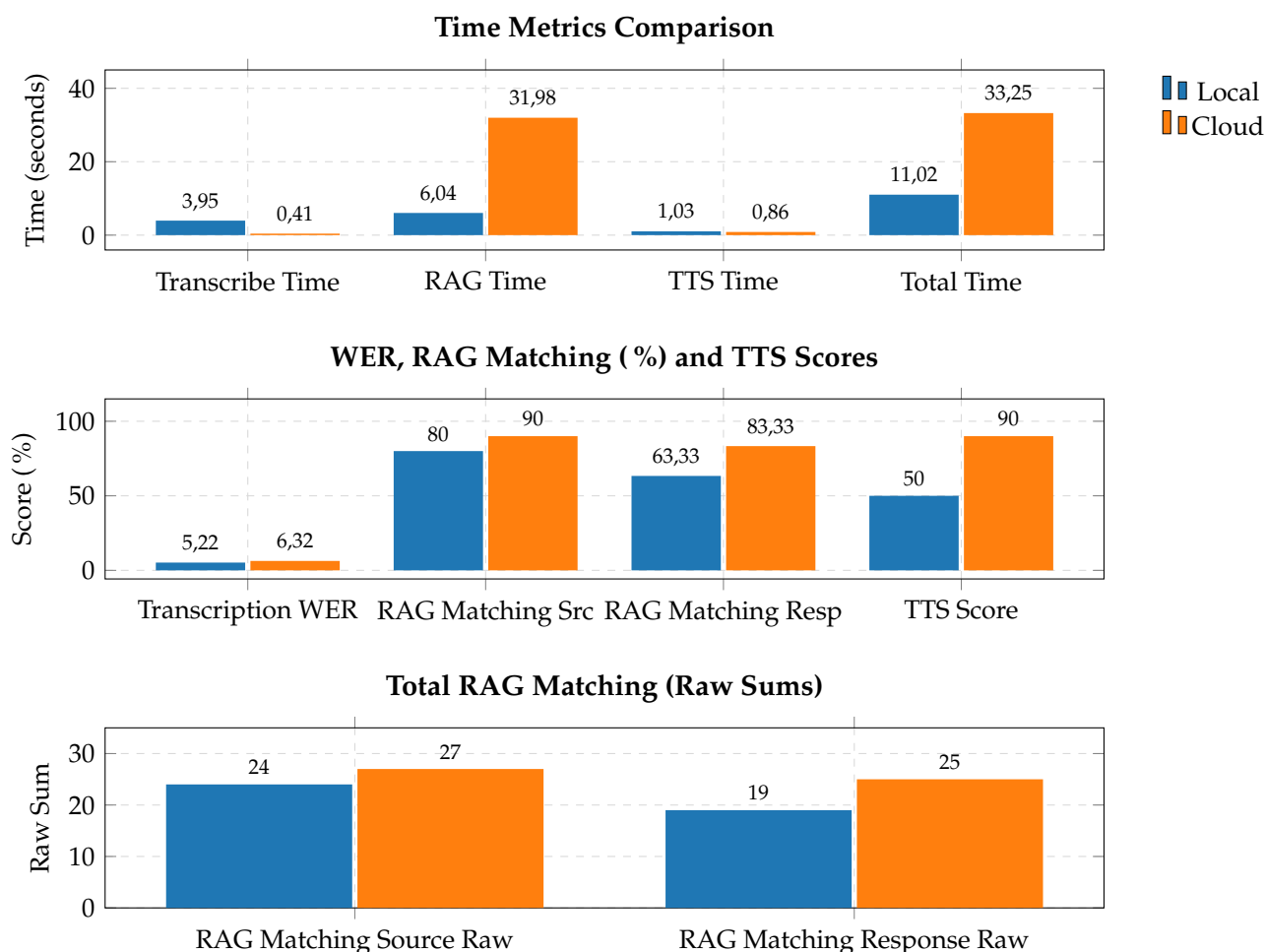


Figura 5.1: Contraste de resultados: tiempos, porcentajes de acierto y conteo crudo.

5.3 Comparativa por fase

- **STT:** Ambas opciones son casi *plug and play*. La nube es $\approx 10\times$ más rápida (0.41s vs 3.95s), pero ninguna representa un cuello de botella crítico. WER prácticamente idéntico (5.22 % local vs 6.32 % cloud). Ambos con la limitación común identificada de siglas y nombres específicos.
- **RAG:** LangChain abstrae por completo la diferencia entre Ollama y los proveedores cloud, permitiendo reutilizar el mismo código cambiando solo la instancia del modelo. La nube supera en precisión final (83.33 % vs 63.33 %) pero es $\approx \times 5$ más lenta en el tiempo de orquestación (31.98s vs 6.04s). El evaluador local (llama3.1) mostró mayor tendencia a descartar contexto válido y derivar búsquedas a la web innecesariamente, especialmente en preguntas sobre figuras conocidas (Messi).

Esta diferencia de precisión se debe a los modelos de embeddings, se ve la diferencia de calidad entre mistral-nemo (local) y sentence-transformers (cloud), que impacta directamente en la recuperación de contexto relevante. Estos factores, al igual que otros, se debería de optimizar para poder hacer uso de este sistema en caso de querer escalar a un entorno de producción.

- **TTS:** AWS supera ampliamente a Kokoro en naturalidad: pausas lógicas, inflexiones y fluidez claramente superiores, con latencia incluso menor (0.86s vs 1.03s). Kokoro es pequeño y eficiente, pero suena plano en comparación.
- **Complejidad de integración:** El entorno local es más simple de desplegar (una sola máquina, sin gestión de API keys ni cuotas). La solución cloud multiproveedor introduce complejidad

burocrática (autenticación, cuotas, políticas de cada servicio) que eleva el esfuerzo de despliegue pese a su mejor rendimiento en calidad.

- **Hardware y privacidad:** Además, hay que tener en cuenta otras consideraciones no cuantificables pero relevantes a la hora de elegir entre una opción u otra: La privacidad de los datos, las dependencias de terceros, la escalabilidad, la flexibilidad para cambiar componentes o la necesidad de hardware específico.

En este sentido: La opción **local** ofrece máxima **privacidad** y control total sobre los datos, **sin depender de terceros** ni de sus políticas. Sin embargo, **requiere hardware** potente (VRAM/RAM) y puede ser más difícil de escalar o mantener. La **nube** externaliza la gestión y el mantenimiento, ofrece **escalabilidad** instantánea, calidad superior en algunos casos (ya que son modelos privados de empresas) y ofrece un **gran catálogo de modelos** (aún que a nivel opensource cada vez tenemos más opciones), pero a costa de la privacidad y la dependencia de proveedores externos.

Sobre todo, destacar la privacidad y la dependencia del entorno local que requiere VRAM y RAM considerables. Sin hardware suficiente, la nube es la única opción viable. A cambio, la nube externaliza los datos a terceros y depende de planes y políticas que pueden cambiar.

5.3.1. Curiosidades y fallos observados

- **STT:** Siglas de dominio específico transcritas incorrectamente por los ASRs, ('MIARFIDS' → 'MirFits', 'PyCatans' → 'PaiKatans') provocan fallos en cadena en el RAG que no permiten recuperar el contexto correcto y comprometen la respuesta final.
- **RAG — fuente incorrecta:** El evaluador ignoraba contexto local válido y forzaba búsqueda web (frecuente en preguntas sobre Messi). La cosa es que de normal, cuando el LLM se queda con contexto insuficiente, alucina la respuesta. En cambio, al salir a la web, aunque no sea lo ideal, es más probable que acierte. Esto se refleja en la diferencia de precisión entre ambos entornos.
- **RAG — inconsistencia web:** Preguntas con respuesta subjetiva o no determinista ('¿raza de perro más peluda?') generaban respuestas distintas en cada ejecución ('Chow Chows', 'Akitas', 'Pullis'...) dependiendo de la fuente consultada por Tavily. Véase Figura 5.2.



Figura 5.2: Inconsistencia del evaluador RAG: prefiere la web ignorando datos locales disponibles.

5.4 Tiempos de respuesta

Como se aprecia en los tiempos de la figura 5.1, el entorno en la nube tiene una latencia de respuesta global muy alta (principalmente por el excesivo tiempo consumido en la orquestación RAG). Pese a que en tareas aisladas como la transcripción de audio (STT) la nube demuestra ser notablemente más rápida frente a la ejecución local, el tiempo final se resiente.

Ya que el feedback del sistema es uno de los factores más críticos para la experiencia del usuario, tardar tanto es un gran inconveniente, de media, el sistema cloud es hasta 3 veces más lento que en el entorno local.

Las diferentes partes terminan dando un resultado similar y al final se consiguen respuestas correctas en ambos casos, pero la espera es un factor que puede llegar a ser molesto para el usuario (más incluso que la calidad de voz por ejemplo).

5.5 Estructura sugerida

Para finalizar, se propone una estructura de sistema híbrida que combine lo mejor de ambos mundos (entre lo que se ha probado): **usar proveedores cloud para el reconocimiento de voz (STT) y la síntesis de voz (TTS) y llevar gestión del RAG de forma local.**

De esta forma, se consigue la latencia más baja (según las pruebas 7,31s, como referencia, Alexa responde en 1 a 4 segundos), se mantiene la máxima privacidad con respecto a los datos procesados, y se obtiene una calidad de voz sobresaliente. Además, el sistema RAG local puede ser tan potente como la máquina lo permita, y no se ve limitado por las restricciones de los proveedores externos.

El único inconveniente de esta arquitectura sería la limitación en la elección de LLMs. Pero con la creciente variedad de modelos, cada vez más pequeños y eficientes, esto no debería ser un gran problema para tareas 'tan simples' como responder preguntas concretas basadas en un contexto específico.

CAPÍTULO 6

Conclusiones y trabajos futuros

Queda demostrado que es totalmente viable desplegar un flujo completo de interacción por voz tanto en local como en la nube, con resultados sólidos en tiempos competentes. Las conclusiones clave por bloque son:

- **STT:** La nube es tiende a ser más rápida (0.41s vs 3.95s) pero con WER prácticamente idéntico (5.22 % local vs 6.32 % cloud). Ambos con limitaciones similares en siglas y nombres específicos.
- **RAG:** La nube ofrece mayor tasa de acierto en respuesta final por el modelo de embeddings (83.33 % vs 63.33 %), pero con latencia disparada y pérdida de privacidad. El evaluador local alucina cuando su contexto es escaso, pero abusar del *fallback* web expone al sistema a información inestable o no determinista. Algo fácilmente arreglable con un modelo de embeddings local más potente, sin tener que recurrir a la nube.
- **TTS:** AWS es ganador indiscutible en naturalidad (score 90 vs 50) con latencia igual o menor que Kokoro. A no ser que la privacidad sea prioritaria, la nube gana en esta fase.
- **Despliegue:** Lo local garantiza privacidad total y simplicidad; la nube aporta un catálogo más potente pero con complejidad burocrática, dependencia de terceros e incertidumbre sobre sus futuras políticas. Ahora sí, los planes gratuitos actuales son suficientes para uso personal. Por lo que sin pagar (ni suscripciones ni hardware), la nube ofrece un sistema completo.

La elección depende de las prioridades: sin hardware potente, la nube es la única opción; con máquina capaz y prioridad en privacidad, el entorno local es preferible; buscando un compromiso, la arquitectura híbrida (STT+TTS cloud, RAG local) es la más adecuada.

6.1 Trabajos futuros

- **Optimización y Streaming:** El sistema más rápido actual ronda los 7s de respuesta. Usar modelos más ligeros y generar texto/audio en tiempo real (*streaming*) reduciría drásticamente la latencia percibida.
- **Wake Word:** Sistema 'siempre escuchando' (estilo 'OK Google') que mantenga el micrófono en bajo consumo y active el flujo pesado solo al detectar la palabra clave.
- **Hardware empotrado:** Portar el sistema a Raspberry Pi o NVIDIA Jetson para un altavoz autónomo, asumiendo que las limitaciones de este hardware obligarán a delegar ciertos servicios a la nube.
- **Ampliación del agente:** Incorporar más herramientas MCP y volúmenes de datos mayores para superar el modo pregunta-respuesta, lo que podría requerir de LLMs más grandes.

Bibliografía

- [1] OpenAI. *Hello GPT-4o*. Accessed: 2026-05-11. 2024. URL: <https://openai.com/index/hello-gpt-4o/>.
- [2] Google. *Gemini Live: A new way to interact with AI*. Accessed: 2026-05-11. 2024. URL: <https://blog.google/products/gemini/made-by-google-gemini-ai-updates/>.
- [3] Alec Radford et al. *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. 2022. URL: <https://github.com/openai/whisper>.
- [4] Meta AI. *Llama 3.1 Model Card*. Accessed: 2026-01-25. Jul. de 2024. URL: <https://huggingface.co/meta-llama/Llama-3.1-8B>.
- [5] Mistral AI y NVIDIA. *Mistral NeMo*. Accessed: 2026-05-10. 2024. URL: <https://mistral.ai/news/mistral-nemo/>.
- [6] hexgrad. *Kokoro-82M: A Low-Parameter TTS Model*. Accessed: 2025-01-25. 2025. URL: <https://huggingface.co/hexgrad/Kokoro-82M/blob/main/VOICES.md#spanish>.
- [7] Qwen Team y Alibaba Cloud. *Qwen3-TTS: Open-source Series of TTS Models*. GitHub Repository. 2025. URL: <https://github.com/QwenLM/Qwen3-TTS?tab=readme-ov-file#environment-setup>.
- [8] LangChain, Inc. *LangChain Documentation*. Accessed: 2025-01-25. 2025. URL: <https://docs.langchain.com/>.
- [9] Tavily AI. *Tavily: The Search Engine for AI Agents*. Accessed: 2026-01-25. 2024. URL: <https://tavily.com/>.
- [10] Snowflake Inc. *Streamlit: A Faster Way to Build and Share Data Apps*. Accessed: 2025-01-25. 2025. URL: <https://streamlit.io/>.
- [11] Jeffrey Morgan y Ollama Team. *Ollama: Get Up and Running with Large Language Models locally*. Accessed: 2026-01-25. 2023. URL: <https://ollama.com/>.
- [12] Groq, Inc. *Groq: The LPU Inference Engine*. Accessed: 2026-05-10. 2025. URL: <https://groq.com/>.
- [13] Hugging Face. *Hugging Face: The AI community building the future*. Accessed: 2026-05-10. 2025. URL: <https://huggingface.co/>.
- [14] Amazon Web Services, Inc. *Amazon Web Services (AWS)*. Accessed: 2026-05-10. 2025. URL: <https://aws.amazon.com/>.
- [15] M. R. U. Islam. *werpy: A powerful and lightweight Python package to calculate Word Error Rate (WER)*. Accessed: 2026-05-10. 2024. URL: <https://pypi.org/project/werpy/>.